

Module 3 In-class Coding Practice - Exception Control

 Published

Java exception control, often referred to as exception handling, is a mechanism that allows developers to manage and respond to exceptional or unexpected situations that may occur during the execution of a Java program. Exceptions are events or conditions that can disrupt the normal flow of a program, and they can be caused by a variety of factors, including runtime errors, user input, or external data sources.

Here's an explanation of the key concepts and components of Java exception control:

1. **Exception:** An exception is an object that represents an exceptional condition or error that occurred during program execution. Exceptions can be of various types, such as `ArithmeticException`, `NullPointerException`, `IOException`, and custom exceptions created by developers.
2. **Exception Handling:** Exception handling is the process of dealing with exceptions when they occur. It allows you to gracefully respond to exceptions, prevent program crashes, and provide meaningful error messages to users.
3. **Try-Catch Block:** The primary mechanism for handling exceptions in Java is the try-catch block. The code that might generate an exception is placed within the "try" block, and the code to handle the exception is placed within one or more "catch" blocks.
4. **Exception Types:** Java has a hierarchy of exception types, with the base class being `java.lang.Throwable`. The two main types of exceptions are:
 - **Checked Exceptions:** These are exceptions that the compiler checks, and you must either catch or declare that your method may throw them (using the "throws" keyword).
 - **Unchecked Exceptions (Runtime Exceptions):** These are exceptions that are not checked at compile-time and typically result from logic errors, such as dividing by zero or accessing a null object reference.
5. **Throwing Exceptions:** Developers can throw exceptions explicitly using the `throw` statement. Custom exceptions can be created by extending the `Exception` class or its subclasses.

0

6. **Finally Block:** In addition to the try-catch block, Java also provides a "finally" block that is used to execute code that should run regardless of whether an exception occurred. This block is often used for resource cleanup.

0

7. **Exception Propagation:** When an exception is thrown in a method, it can be caught and handled in the current method or propagated to higher-level methods. If an exception is not caught, it will continue to propagate up the call stack until it is caught or the program terminates.

8. **Handling Multiple Exceptions:** Multiple catch blocks can be used to handle different types of exceptions in the same try-catch block. They are evaluated from top to bottom, and the first matching catch block is executed.

0

9. **Best Practices:** When handling exceptions, it's essential to provide meaningful error messages, log exceptions, and follow good coding practices to minimize the risk of exceptions. Handling exceptions gracefully can improve the reliability and robustness of your Java applications.

Java exception control allows developers to write more robust and reliable programs by addressing exceptional conditions, preventing crashes, and improving error reporting. It's an essential part of writing production-ready software in Java.

Example 1: Handling `ArithmeticException` (Skeleton Code)

```
public class ArithmeticExceptionExample {  
    public static void main(String[] args) {  
        try {  
            // Call a method that may throw an ArithmeticException  
            // Handle the exception with a catch block  
        } catch (ArithmeticException e) {  
            // Handle the exception and display an error message  
        }  
    }  
  
    public static int divide(int dividend, int divisor) {  
        // Implement a method that may throw an ArithmeticException  
    }  
}
```

Example 2: Custom Exception (Skeleton Code)

```
class CustomException extends Exception {  
    // Define a custom exception class  
}  
  
public class CustomExceptionExample {  
    public static void main(String[] args) {  
        try {  
            // Call a method that may throw a CustomException  
            // Handle the CustomException with a catch block  
        } catch (CustomException e) {  
            // Handle the CustomException and display an error message  
        }  
    }  
  
    public static void validateAge(int age) throws CustomException {  
        // Implement a method that may throw a CustomException  
    }  
}
```

Example 3: Multiple Exceptions (Skeleton Code)

Grading Rubric

Example 1: Handling ArithmeticException (Maximum 35 Points)

Criteria

Points

Code Structure

Criteria**Points**

- Proper use of try-catch block with a method that may throw an `ArithmeticException` 5

Implementation and Output

- Proper exception handling with a meaningful error message 10
- Code correctly demonstrates handling an `ArithmeticException` 10

Code Comments and Naming

- Clear comments in classes and methods to describe their purpose 5
- Code follows naming conventions for variables, methods, and classes 5

Total Points (Example 1) 35

Example 2: Custom Exception (Maximum 35 Points)**Criteria****Points**

Code Structure

- Proper use of custom exception and try-catch block 5

Implementation and Output

- Proper exception handling with a meaningful error message 10
- Code correctly demonstrates handling a custom exception 10

Code Comments and Naming

- Clear comments in classes and methods to describe their purpose 5
- Code follows naming conventions for variables, methods, and classes 5

Total Points (Example 2) 35

Example 3: Multiple Exceptions (Maximum 30 Points)**Criteria****Points**

Code Structure

- Proper use of multiple catch blocks for different exceptions 5

Implementation and Output

Criteria	Points
- Proper exception handling with meaningful error messages	10
- Code correctly demonstrates handling multiple exceptions	10
Code Comments and Naming	
- Clear comments in classes and methods to describe their purpose	5
- Code follows naming conventions for variables, methods, and classes	5
Total Points (Example 3)	30

The maximum achievable score for the combined examples is 100 points. The rubric assesses the code based on various criteria, including code structure, implementation correctness, documentation, and adherence to best practices in exception handling.

Points 100

Submitting a file upload

Due	For	Available from	Until
Sep 30	Everyone	-	Oct 22 at 11:59pm